

A PROJECT SOURCE CODE ON

COVID-19 DATA VISUALIZATION

(Data Analytics & Business Intelligence)

Project Domain	Data Analytics & Business Intelligence
Project Type	Data Visualization and Interactive Dashboard
Department	Computer Science and Engineering (Artificial Intelligence and Machine Learning)
Primary Objective	Analysis and visual representation of COVID-19 data

SOURCE CODE

This document contains the implementation-oriented source code for the COVID-19 Data Visualization project. The organization follows the supplied academic sample's System Development structure, including project design, requirements, technologies, folder structure, data processing, pseudocode and application flow.

1. Project Architecture

DATA SOURCE	→	PREPROCESSING	→	ANALYSIS
CSV / Excel	→	Cleaning + Transformation	→	KPIs + Aggregation
DASHBOARD	←	VISUALIZATION	←	FILTERS

2. Functional Requirements

#	Requirement	Implementation
FR1	Data Import	Read COVID-19 CSV/Excel data using Pandas.
FR2	Data Cleaning	Handle missing values, duplicates and invalid numeric values.
FR3	Transformation	Convert dates and create year/month fields.
FR4	KPI Calculation	Calculate cases, deaths, recoveries and active cases.
FR5	Filtering	Filter by date and location when available.
FR6	Visualization	Generate trend and comparison charts.
FR7	Dashboard	Display KPIs, charts, preview and download.

3. Technologies Used

- Python — main programming language.
- Pandas — data loading, cleaning and transformation.
- Plotly — interactive charts.
- Streamlit — dashboard interface.
- NumPy — numerical operations.

Chapter 03: SYSTEM DEVELOPMENT

3.1 Complete Source Code — app.py

```
import os
import numpy as np
import pandas as pd
import plotly.express as px
import streamlit as st

st.set_page_config(
    page_title="COVID-19 Data Visualization",
    page_icon="📊",
    layout="wide"
)

DATA_PATH = "data/covid19_dataset.csv"

def load_data(path):
    if not os.path.exists(path):
        raise FileNotFoundError(
            f"Dataset not found: {path}"
        )
    df = pd.read_csv(path)
    df.columns = (
        df.columns.astype(str).str.strip().str.lower()
        .str.replace(" ", "_").str.replace("-", "_")
        .str.replace("/", "_")
    )
    return df

def find_column(df, candidates):
    for name in candidates:
        if name in df.columns:
            return name
    return None

def prepare_data(df):
    date_col = find_column(
        df, ["date", "observation_date", "reported_date"])
    country_col = find_column(
        df, ["country", "country_region", "country_or_region",
            "location", "region"])
    confirmed_col = find_column(
        df, ["confirmed", "confirmed_cases", "total_cases",
            "cases", "case_count"])
    deaths_col = find_column(
        df, ["deaths", "death", "total_deaths", "death_count"])
    recovered_col = find_column(
        df, ["recovered", "recoveries", "total_recovered"])
    active_col = find_column(
        df, ["active", "active_cases", "active_case"])

    if date_col:
        df[date_col] = pd.to_datetime(
            df[date_col], errors="coerce")
        df = df.dropna(subset=[date_col])
        df["year"] = df[date_col].dt.year
        df["month"] = df[date_col].dt.to_period("M").astype(str)

    for col in [confirmed_col, deaths_col,
                recovered_col, active_col]:
        if col:
            df[col] = pd.to_numeric(
                df[col], errors="coerce").fillna(0)

    df = df.drop_duplicates().reset_index(drop=True)

    return df, {
        "date": date_col,
        "country": country_col,
        "confirmed": confirmed_col,
        "deaths": deaths_col,
        "recovered": recovered_col,
        "active": active_col
    }
```

3.2 KPI Helper Functions

```
def latest_value(df, column, date_col):
    if column is None or date_col is None:
        return 0
    latest_date = df[date_col].max()
    return float(
        df.loc[df[date_col] == latest_date, column].sum()
    )

def format_number(value):
    return f"{value:,.0f}"

def build_kpis(df, cols):
    return {
        "confirmed": latest_value(
            df, cols["confirmed"], cols["date"]),
        "deaths": latest_value(
            df, cols["deaths"], cols["date"]),
        "recovered": latest_value(
            df, cols["recovered"], cols["date"]),
        "active": latest_value(
            df, cols["active"], cols["date"])
    }
```

3.3 Dashboard Interface and Filters

```

try:
    raw_df = load_data(DATA_PATH)
    df, cols = prepare_data(raw_df)
except Exception as exc:
    st.error(str(exc))
    st.stop()

st.title("COVID-19 Data Visualization")
st.caption(
    "Computer Science and Engineering – "
    "Artificial Intelligence and Machine Learning"
)

with st.sidebar:
    st.header("Dashboard Filters")

    if cols["date"]:
        min_date = df[cols["date"]].min().date()
        max_date = df[cols["date"]].max().date()

        selected_dates = st.date_input(
            "Date Range",
            value=(min_date, max_date),
            min_value=min_date,
            max_value=max_date
        )

        if isinstance(selected_dates, tuple) and len(selected_dates) == 2:
            start_date, end_date = selected_dates
            df = df[
                (df[cols["date"]].dt.date >= start_date) &
                (df[cols["date"]].dt.date <= end_date)
            ]

    if cols["country"]:
        locations = sorted(
            df[cols["country"]].dropna().astype(str).unique()
        )
        selected_locations = st.multiselect(
            "Location", locations, default=locations)

        if selected_locations:
            df = df[
                df[cols["country"]].astype(str)
                .isin(selected_locations)
            ]

kpis = build_kpis(df, cols)

c1, c2, c3, c4 = st.columns(4)
with c1:
    st.metric("Confirmed Cases",
              format_number(kpis["confirmed"]))
with c2:
    st.metric("Deaths", format_number(kpis["deaths"]))
with c3:
    st.metric("Recoveries",
              format_number(kpis["recovered"]))
with c4:
    st.metric("Active Cases",
              format_number(kpis["active"]))

```

3.4 Time-Series Visualization

```

if cols["date"] and cols["confirmed"]:
    trend = (
        df.groupby(cols["date"], as_index=False)[cols["confirmed"]]
        .sum().sort_values(cols["date"])
    )

    fig = px.line(
        trend, x=cols["date"], y=cols["confirmed"],
        title="COVID-19 Confirmed Cases Over Time",
        markers=True
    )
    fig.update_layout(
        xaxis_title="Date",
        yaxis_title="Confirmed Cases",
        hovermode="x unified"
    )
    st.plotly_chart(fig, use_container_width=True)
else:
    st.info(
        "Date and confirmed-case columns are required."
    )

```

3.5 Additional Visualizations

Death and Recovery Trends

```
trend_columns = []

if cols["deaths"]:
    trend_columns.append((cols["deaths"], "Deaths Over Time"))

if cols["recovered"]:
    trend_columns.append(
        (cols["recovered"], "Recoveries Over Time"))

for value_col, chart_title in trend_columns:
    if cols["date"]:
        trend = (
            df.groupby(cols["date"], as_index=False)[value_col]
            .sum().sort_values(cols["date"])
        )
        fig = px.line(
            trend, x=cols["date"], y=value_col,
            title=chart_title, markers=True
        )
        fig.update_layout(
            xaxis_title="Date",
            yaxis_title=value_col.replace("_", " ").title(),
            hovermode="x unified"
        )
    st.plotly_chart(fig, use_container_width=True)
```

Regional / Location Comparison

```
if cols["country"] and cols["confirmed"]:
    comparison = (
        df.groupby(cols["country"], as_index=False)[cols["confirmed"]]
        .sum()
        .sort_values(cols["confirmed"], ascending=False)
        .head(10)
    )

    fig = px.bar(
        comparison,
        x=cols["country"],
        y=cols["confirmed"],
        title="Top Locations by Confirmed Cases",
        text_auto=True
    )
    fig.update_layout(
        xaxis_title="Location",
        yaxis_title="Confirmed Cases",
        xaxis_tickangle=-35
    )
    st.plotly_chart(fig, use_container_width=True)
else:
    st.info("Location and confirmed-case columns are required.")
```

Dataset Preview and Download

```
st.subheader("Dataset Preview")
st.dataframe(df.head(100), use_container_width=True)

csv_output = df.to_csv(index=False).encode("utf-8")

st.download_button(
    label="Download Cleaned Dataset",
    data=csv_output,
    file_name="cleaned_covid19_dataset.csv",
    mime="text/csv"
)

st.success(
    "COVID-19 data analysis and visualization completed successfully."
)
```

3.6 Supporting Files and Execution

requirements.txt

```
pandas>=2.0
numpy>=1.24
plotly>=5.15
streamlit>=1.30
```

Data Preparation Script

```
import pandas as pd

INPUT_FILE = "data/covid19_dataset.csv"
OUTPUT_FILE = "data/cleaned_covid19_dataset.csv"

df = pd.read_csv(INPUT_FILE)

df.columns = (
    df.columns.astype(str).str.strip().str.lower()
    .str.replace(" ", "_").str.replace("-", "_")
)

if "date" in df.columns:
    df["date"] = pd.to_datetime(
        df["date"], errors="coerce"
    )
    df = df.dropna(subset=["date"])
    df["year"] = df["date"].dt.year
    df["month"] = df["date"].dt.to_period("M").astype(str)

numeric_fields = [
    "confirmed", "confirmed_cases", "deaths",
    "total_deaths", "recovered", "recoveries",
    "active", "active_cases"
]

for col in numeric_fields:
    if col in df.columns:
        df[col] = pd.to_numeric(
            df[col], errors="coerce").fillna(0)

df = df.drop_duplicates()
df.to_csv(OUTPUT_FILE, index=False)

print("Cleaned dataset saved to:", OUTPUT_FILE)
```

Execution Commands

```
python -m venv venv

# Windows
venv\Scripts\activate

pip install -r requirements.txt

streamlit run app.py
```

Project Folder Structure

```
COVID-19-Data-Visualization/
├── data/
│   ├── covid19_dataset.csv
│   ├── app.py
│   ├── requirements.txt
│   ├── README.md
│   └── docs/
│       └── Source_Code.pdf
```

Source Code Notes

- The code recognizes common alternative names for date, location, confirmed, deaths, recoveries and active-case columns.
- The exact record count and final feature list must be taken from the dataset actually used in the project.
- Deaths, recoveries and active cases are optional; unavailable fields are skipped.
- The dashboard is a project-scale visualization system. Predictive ML is a possible future extension.
- The source code implements the complete data-to-dashboard workflow: loading → cleaning → transformation → KPI calculation → visualization → dashboard.

Conclusion

The source code provides a reproducible implementation of the COVID-19 Data Visualization project, combining data preparation, KPI analysis and interactive visualization. It is structured for academic submission and can be extended with forecasting, anomaly detection, automated refresh and additional visual analytics.